JADAVPUR UNIVERSITY

DSA Lab

Course: MCA 2026-27

Name: Anubhav Chakraborty

Roll No: 002510503021

Semester: 2nd

Data Structures Assignments

Index

Sl. No.	Problem	Page
1	Assignment 1 : Problem 1	3
2	Assignment 2 : Problem 1	11
3	Assignment 2 : Problem 2	24
4	Assignment 2 : Problem 3	30
5	Assignment 2 : Problem 4	39
6	Assignment 3 : Problem 1	43
7	Assignment 3 : Problem 2	46
8	Assignment 3 : Problem 3	49
9	Assignment 3 : Problem 4	50
10	Assignment 3 : Problem 5	51
11	Assignment 3 : Problem 6	54
12	Assignment 3 : Problem 7	56
13	Assignment 3 : Problem 8	58
14	Assignment 3 : Problem 9	60
15	Assignment 3 : Problem 10	64
16	Assignment 3 : Problem 11	68
17	Assignment 3 : Problem 12	72
18	Assignment 3 : Problem 13	76
19	Assignment 3 : Problem 14	80
20	Assignment 3 : Problem 15	83
21	Assignment 3 : Problem 16	87
22	Assignment 3 : Problem 17	91
23	Assignment 3 : Problem 18	96

Assignment 1 : Problem 1

Problem Statement:
Write a menu-driven program in C to perform the following operations in an integer array allocated memory dynamically. The list may grow or shrink compared to the initial allocation as and when required. Multiple operations may be performed on the existing list without recompiling/reexecuting the program. The individual operation should be implemented using a single function. List of operations:

a) Create
b) Display the entire list
c) Count (total number of elements in the list)
d) Reverse the list (reversed list should be stored in the same array
e) Index of a given elemens (all indices if there are duplicate entries
f) Indexed element
g) Insert
h) Delete
i) Merge
j) Split
k) Sort

ass-1.c

```c
#include <stdio.h>
#include <stdlib.h>

int* arr = NULL;
int size = 0;

void create();
void display();
void countElements();
void reverse();
void searchElement();
void indexedElement();
void insertElement();
void deleteElement();
void mergeArrays();
void splitArray();
void sortArray();

int main ()
{
  int c;

  while (1)
  {
    printf("\nMENU:\n1. Create\n2. Display\n3. Count\n4. Reverse\n5. Index of Element\n6.
Indexed Element\n7. Insert\n8. Delete\n9. Merge\n10. Split\n11. Sort\n12. Exit\nEnter your
choice: ");
    scanf("%d", &c);

    switch(c) {
      case 1: create(); break;
      case 2: display(); break;
      case 3: countElements(); break;
      case 4: reverse(); break;
      case 5: searchElement(); break;
      case 6: indexedElement(); break;
      case 7: insertElement(); break;
      case 8: deleteElement(); break;
```

```c
      case 9: mergeArrays(); break;
      case 10: splitArray(); break;
      case 11: sortArray(); break;
      case 12: free(arr); printf("Program Exited.\n"); return 0;
      default: printf("Invalid choice!\n");
    }
  }
  return 0;
}

void create() {
    int i;

    printf("Enter number of elements: ");
    scanf("%d", &size);

    arr = (int *)malloc(size * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    printf("Enter elements:\n");
    for (i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Array created successfully.\n");
}

void display() {
    int i;

    if (size == 0) {
        printf("Array is empty.\n");
        return;
    }

    printf("Array elements are:\n");

    for (i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }

    printf("\n");
}

void countElements() {
    printf("Total number of elements = %d\n", size);
}

void reverse() {
    int i, temp;

    for (i = 0; i < size / 2; i++) {
        temp = arr[i];
        arr[i] = arr[size - i - 1];
        arr[size - i - 1] = temp;
    }

    printf("Array reversed successfully.\n");
}
```

```c
void searchElement() {
    int key, i, found = 0;

    printf("Enter element to search: ");
    scanf("%d", &key);

    printf("Element found at index: ");

    for (i = 0; i < size; i++) {
        if (arr[i] == key) {
            printf("%d ", i);
            found = 1;
    }
  }

  if (!found) {
    printf("Not found");
  }

  printf("\n");
}

void indexedElement() {
    int index;
    if (size == 0) {
        printf("Array is empty.\n");
        return;
    }
    printf("Enter index (0 to %d): ", size - 1);
    scanf("%d", &index);

    if (index >= 0 && index < size) {
        printf("Element at index %d is: %d\n", index, arr[index]);
    } else {
        printf("Invalid index!\n");
    }
}

void insertElement() {
    int pos, elem, i;
    printf("Enter position to insert (0 to %d): ", size);
    scanf("%d", &pos);

    printf("Enter element to insert: ");
    scanf("%d", &elem);

    if (pos < 0 || pos > size) {
        printf("Invalid position!\n");
        return;
    }

    int* temp = (int*)realloc(arr, (size + 1) * sizeof(int));
    if (temp == NULL) {
        printf("Memory allocation failed!\n");
        return;
    }
    arr = temp;

    for (i = size; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
```

```
    arr[pos] = elem;
    size++;
    printf("Element inserted successfully.\n");
}

void deleteElement() {
    int pos, i;
    if (size == 0) {
        printf("Array is empty. Nothing to delete.\n");
        return;
    }
    printf("Enter position to delete (0 to %d): ", size - 1);
    scanf("%d", &pos);

    if (pos < 0 || pos >= size) {
        printf("Invalid position!\n");
        return;
    }

    for (i = pos; i < size - 1; i++) {
        arr[i] = arr[i + 1];
    }

    size--;

    if (size > 0) {
        arr = (int*)realloc(arr, size * sizeof(int));
    } else {
        free(arr);
        arr = NULL;
    }
    printf("Element deleted successfully.\n");
}

void mergeArrays() {
    int size2, i;
    printf("Enter size of the second array to merge: ");
    scanf("%d", &size2);

    if (size2 <= 0) {
        printf("Invalid size.\n");
        return;
    }

    int* arr2 = (int*)malloc(size2 * sizeof(int));
    if (arr2 == NULL) {
        printf("Memory allocation failed for second array!\n");
        return;
    }

    printf("Enter elements of second array:\n");
    for (i = 0; i < size2; i++) {
        scanf("%d", &arr2[i]);
    }

    int* temp = (int*)realloc(arr, (size + size2) * sizeof(int));
    if (temp == NULL) {
        printf("Memory allocation failed during merge!\n");
        free(arr2);
        return;
    }
    arr = temp;
```

```
    for (i = 0; i < size2; i++) {
        arr[size + i] = arr2[i];
    }

    size += size2;
    free(arr2);
    printf("Arrays merged successfully.\n");
}

void splitArray() {
    int splitIdx, i;
    if (size <= 1) {
        printf("Array too small to split.\n");
        return;
    }
    printf("Enter index to split from (1 to %d): ", size - 1);
    scanf("%d", &splitIdx);

    if (splitIdx <= 0 || splitIdx >= size) {
        printf("Invalid split index!\n");
        return;
    }

    printf("First Part: ");
    for (i = 0; i < splitIdx; i++) {
        printf("%d ", arr[i]);
    }
    printf("\nSecond Part: ");
    for (i = splitIdx; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    size = splitIdx;
    arr = (int*)realloc(arr, size * sizeof(int));
    printf("Global array resized to keep the first part.\n");
}

void sortArray() {
    int i, j, temp;
    if (size <= 1) {
        printf("Array is already sorted.\n");
        return;
    }

    for (i = 0; i < size - 1; i++) {
        for (j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    printf("Array sorted in ascending order.\n");
}
```

Output

```
$ gcc ass-1.c
$ ./a.out
```

```
MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 1
Enter number of elements: 5
Enter elements:
5 3 8 1 9
Array created successfully.

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 2
Array elements are:
5 3 8 1 9

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 3
Total number of elements = 5

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 4
Array reversed successfully.

MENU:
1. Create
2. Display
3. Count
4. Reverse
```

```
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 2
Array elements are:
9 1 8 3 5

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 5
Enter element to search: 3
Element found at index: 3

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 7
Enter position to insert (0 to 5): 2
Enter element to insert: 100
Element inserted successfully.

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 2
Array elements are:
9 1 100 8 3 5

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
```

```
10. Split
11. Sort
12. Exit
Enter your choice: 8
Enter position to delete (0 to 5): 0
Element deleted successfully.

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 2
Array elements are:
1 100 8 3 5

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 11
Array sorted in ascending order.

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 2
Array elements are:
1 3 5 8 100

MENU:
1. Create
2. Display
3. Count
4. Reverse
5. Index of Element
6. Indexed Element
7. Insert
8. Delete
9. Merge
10. Split
11. Sort
12. Exit
Enter your choice: 12
Program Exited.
```

—

Assignment 2 : Problem 1

Problem Statement:
Write a menu-driven program that performs the following on a single linked list. (Write a function for each of the operations)

I. Create a linked list.
II. Print the content of the list.
III. Insert an element at the front of the list
IV. Insert an element at the end of the list
V. Insert a node after the kth node.
VI. Insert a node after the node (first from the start) containing a value.
VII. Insert a node before the kth node.
VIII. Insert a node before the node (first from the start) containing a given value.
IX. Delete the first node.
X. Delete the last node.
XI. Delete a node after the kth node.
XII. Delete a node before the kth node.
XIII. Delete the kth node.
XIV. Delete the node(first from the start) containing a specified value.
XV. Find the reverse of a list(not just printing in reverse)
XVI. Sort the list
XVII. Search a given element
XVIII. Merge two lists; those are in ascending order.
XIX. Concatenate two list
XX. Find if two lists are equal(Boolean output)

ass-2-1.c

```c
#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* createNode(int data) {
    Node* temp = (Node*)malloc(sizeof(Node));
    temp->data = data;
    temp->next = NULL;
    return temp;
}

void printList(Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    Node* temp = head;
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}
```

```c
void createList(Node** head) {
    int n, val;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter value: ");
        scanf("%d", &val);

        Node* newNode = createNode(val);

        if (*head == NULL) {
            *head = newNode;
        } else {
            Node* temp = *head;
            while (temp->next != NULL)
                temp = temp->next;

            temp->next = newNode;
        }
    }
}

void insertFront(Node** head, int val) {
    Node* newNode = createNode(val);
    newNode->next = *head;
    *head = newNode;
}

void insertEnd(Node** head, int val) {
    Node* newNode = createNode(val);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    Node* temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

void insertAfterK(Node** head, int k, int val) {
    Node* temp = *head;

    for (int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) {
        printf("Invalid position.\n");
        return;
    }

    Node* newNode = createNode(val);
    newNode->next = temp->next;
    temp->next = newNode;
}

void insertAfterValue(Node** head, int key, int val) {
```

```
    Node* temp = *head;

    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL) {
        printf("Value not found.\n");
        return;
    }

    Node* newNode = createNode(val);
    newNode->next = temp->next;
    temp->next = newNode;
}

void insertBeforeK(Node** head, int k, int val) {
    if (k == 1) {
        insertFront(head, val);
        return;
    }

    Node* temp = *head;

    for (int i = 1; i < k - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL || temp->next == NULL) {
        printf("Invalid position.\n");
        return;
    }

    Node* newNode = createNode(val);
    newNode->next = temp->next;
    temp->next = newNode;
}

void insertBeforeValue(Node** head, int key, int val) {
    if (*head == NULL)
        return;

    if ((*head)->data == key) {
        insertFront(head, val);
        return;
    }

    Node* temp = *head;

    while (temp->next != NULL && temp->next->data != key)
        temp = temp->next;

    if (temp->next == NULL) {
        printf("Value not found.\n");
        return;
    }

    Node* newNode = createNode(val);
    newNode->next = temp->next;
    temp->next = newNode;
}

void deleteFirst(Node** head) {
    if (*head == NULL)
        return;
```

```
    Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}

void deleteLast(Node** head) {
    if (*head == NULL)
        return;

    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }

    Node* temp = *head;

    while (temp->next->next != NULL)
        temp = temp->next;

    free(temp->next);
    temp->next = NULL;
}

void deleteAfterK(Node** head, int k) {
    Node* temp = *head;

    for (int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL || temp->next == NULL) {
        printf("Deletion not possible.\n");
        return;
    }

    Node* del = temp->next;
    temp->next = del->next;
    free(del);
}

void deleteBeforeK(Node** head, int k) {
    if (k <= 2 || *head == NULL)
        return;

    Node* temp = *head;

    for (int i = 1; i < k - 2 && temp->next != NULL; i++)
        temp = temp->next;

    Node* del = temp->next;
    temp->next = del->next;
    free(del);
}

void deleteK(Node** head, int k) {
    if (k == 1) {
        deleteFirst(head);
        return;
    }

    Node* temp = *head;
```

```
    for (int i = 1; i < k - 1 && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL || temp->next == NULL) {
        printf("Invalid position.\n");
        return;
    }

    Node* del = temp->next;
    temp->next = del->next;
    free(del);
}

void deleteValue(Node** head, int key) {
    if (*head == NULL)
        return;

    if ((*head)->data == key) {
        deleteFirst(head);
        return;
    }

    Node* temp = *head;

    while (temp->next != NULL && temp->next->data != key)
        temp = temp->next;

    if (temp->next == NULL) {
        printf("Value not found.\n");
        return;
    }

    Node* del = temp->next;
    temp->next = del->next;
    free(del);
}

void reverseList(Node** head) {
    Node *prev = NULL, *curr = *head, *next;

    while (curr != NULL) {
        next = curr->next;
        curr->next = prev;
        prev = curr;
        curr = next;
    }

    *head = prev;
}

void sortList(Node* head) {
    Node *i, *j;
    int temp;

    for (i = head; i != NULL; i = i->next) {
        for (j = i->next; j != NULL; j = j->next) {
            if (i->data > j->data) {
                temp = i->data;
                i->data = j->data;
                j->data = temp;
            }
        }
    }
```

```
}

void search(Node* head, int key) {
    int pos = 1;

    while (head != NULL) {
        if (head->data == key) {
            printf("Element found at position %d\n", pos);
            return;
        }

        head = head->next;
        pos++;
    }

    printf("Element not found.\n");
}

Node* mergeLists(Node* l1, Node* l2) {
    Node dummy;
    Node* tail = &dummy;

    dummy.next = NULL;

    while (l1 != NULL && l2 != NULL) {
        if (l1->data < l2->data) {
            tail->next = l1;
            l1 = l1->next;
        } else {
            tail->next = l2;
            l2 = l2->next;
        }

        tail = tail->next;
    }

    if (l1 != NULL)
        tail->next = l1;
    else
        tail->next = l2;

    return dummy.next;
}

void concatenate(Node** l1, Node* l2) {
    if (*l1 == NULL) {
        *l1 = l2;
        return;
    }

    Node* temp = *l1;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = l2;
}

int areEqual(Node* l1, Node* l2) {
    while (l1 != NULL && l2 != NULL) {
        if (l1->data != l2->data)
            return 0;
```

```
        l1 = l1->next;
        l2 = l2->next;
    }

    return (l1 == NULL && l2 == NULL);
}

int main() {
    Node *list1 = NULL, *list2 = NULL, *merged = NULL;

    int choice, val, k, key;

    while (1) {
        printf("\n--- MENU ---\n");
        printf("1. Create List 1\n");
        printf("2. Print List 1\n");
        printf("3. Insert Front\n");
        printf("4. Insert End\n");
        printf("5. Insert After kth Node\n");
        printf("6. Insert After Value\n");
        printf("7. Insert Before kth Node\n");
        printf("8. Insert Before Value\n");
        printf("9. Delete First\n");
        printf("10. Delete Last\n");
        printf("11. Delete After kth Node\n");
        printf("12. Delete Before kth Node\n");
        printf("13. Delete kth Node\n");
        printf("14. Delete By Value\n");
        printf("15. Reverse List\n");
        printf("16. Sort List\n");
        printf("17. Search Element\n");
        printf("18. Merge Two Sorted Lists\n");
        printf("19. Concatenate Two Lists\n");
        printf("20. Compare Two Lists\n");
        printf("0. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {

        case 1:
            list1 = NULL;
            createList(&list1);
            break;

        case 2:
            printList(list1);
            break;

        case 3:
            printf("Enter value: ");
            scanf("%d", &val);
            insertFront(&list1, val);
            break;

        case 4:
            printf("Enter value: ");
            scanf("%d", &val);
            insertEnd(&list1, val);
            break;

        case 5:
```

```
            printf("Enter k and value: ");
            scanf("%d%d", &k, &val);
            insertAfterK(&list1, k, val);
            break;

        case 6:
            printf("Enter key and value: ");
            scanf("%d%d", &key, &val);
            insertAfterValue(&list1, key, val);
            break;

        case 7:
            printf("Enter k and value: ");
            scanf("%d%d", &k, &val);
            insertBeforeK(&list1, k, val);
            break;

        case 8:
            printf("Enter key and value: ");
            scanf("%d%d", &key, &val);
            insertBeforeValue(&list1, key, val);
            break;

        case 9:
            deleteFirst(&list1);
            break;

        case 10:
            deleteLast(&list1);
            break;

        case 11:
            printf("Enter k: ");
            scanf("%d", &k);
            deleteAfterK(&list1, k);
            break;

        case 12:
            printf("Enter k: ");
            scanf("%d", &k);
            deleteBeforeK(&list1, k);
            break;

        case 13:
            printf("Enter k: ");
            scanf("%d", &k);
            deleteK(&list1, k);
            break;

        case 14:
            printf("Enter value: ");
            scanf("%d", &key);
            deleteValue(&list1, key);
            break;

        case 15:
            reverseList(&list1);
            break;

        case 16:
            sortList(list1);
            break;
```

```
        case 17:
            printf("Enter element to search: ");
            scanf("%d", &key);
            search(list1, key);
            break;

        case 18:
            printf("Create second sorted list:\n");
            createList(&list2);
            sortList(list1);
            sortList(list2);

            merged = mergeLists(list1, list2);
            list2 = NULL;
            list1 = merged;

            printf("Merged List:\n");
            printList(merged);
            break;

        case 19:
            printf("Create second list:\n");
            createList(&list2);

            concatenate(&list1, list2);
            list2 = NULL;

            printf("Concatenated List:\n");
            printList(list1);
            break;

        case 20:
            printf("Create second list:\n");
            createList(&list2);

            if (areEqual(list1, list2))
                printf("Lists are equal.\n");
            else
                printf("Lists are not equal.\n");

            break;

        case 0:
            exit(0);

        default:
            printf("Invalid choice.\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-2-1.c
$ ./a.out

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
```

```
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
Enter choice: 1
Enter number of nodes: 5
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Enter value: 50

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
Enter choice: 2
10 -> 20 -> 30 -> 40 -> 50 -> NULL

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
Enter choice: 3
```

```
Enter value: 99

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
Enter choice: 2
99 -> 10 -> 20 -> 30 -> 40 -> 50 -> NULL

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
Enter choice: 17
Enter element to search: 30
Element found at position 4

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
```

```
0. Exit
Enter choice: 16

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
Enter choice: 2
10 -> 20 -> 30 -> 40 -> 50 -> 99 -> NULL

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
Enter choice: 15

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
```

```
Enter choice: 2
99 -> 50 -> 40 -> 30 -> 20 -> 10 -> NULL

--- MENU ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
16. Sort List
17. Search Element
18. Merge Two Sorted Lists
19. Concatenate Two Lists
20. Compare Two Lists
0. Exit
Enter choice: 0
```

—

Assignment 2 : Problem 2

Problem Statement:
Write a menu-driven program representing a polynomial as a data structure using a singly linked list and write functions to add, subtract and multiply two polynomials.

ass-2-2.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int coeff;
    int exp;
    struct Node* next;
} Node;

Node* createNode(int coeff, int exp) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    newNode->coeff = coeff;
    newNode->exp = exp;
    newNode->next = NULL;
    return newNode;
}

void clearPolynomial(Node** headRef) {
    Node* current = *headRef;
    while (current != NULL) {
        Node* nextNode = current->next;
        free(current);
        current = nextNode;
    }
    *headRef = NULL;
}

void insertTerm(Node** headRef, int coeff, int exp) {
    if (coeff == 0) return;

    if (*headRef == NULL || exp > (*headRef)->exp) {
        Node* newNode = createNode(coeff, exp);
        newNode->next = *headRef;
        *headRef = newNode;
        return;
    }

    if ((*headRef)->exp == exp) {
        (*headRef)->coeff += coeff;
        if ((*headRef)->coeff == 0) {
            Node* temp = *headRef;
            *headRef = (*headRef)->next;
            free(temp);
        }
        return;
    }

    Node* current = *headRef;
    while (current->next != NULL && current->next->exp > exp) {
```

```
        current = current->next;
    }

    if (current->next != NULL && current->next->exp == exp) {
        current->next->coeff += coeff;
        if (current->next->coeff == 0) {
            Node* temp = current->next;
            current->next = current->next->next;
            free(temp);
        }
    } else {
        Node* newNode = createNode(coeff, exp);
        newNode->next = current->next;
        current->next = newNode;
    }
}

void displayPolynomial(Node* head) {
    if (head == NULL) {
        printf("0\n");
        return;
    }

    Node* current = head;
    int isFirst = 1;

    while (current != NULL) {
        if (current->coeff > 0 && !isFirst) {
            printf(" + ");
        } else if (current->coeff < 0) {
            printf(isFirst ? "-" : " - ");
        }

        int absCoeff = abs(current->coeff);

        if (absCoeff != 1 || current->exp == 0) {
            printf("%d", absCoeff);
        }

        if (current->exp > 0) {
            printf("x");
            if (current->exp > 1) {
                printf("^%d", current->exp);
            }
        }

        isFirst = 0;
        current = current->next;
    }
    printf("\n");
}

Node* addPolynomials(Node* p1, Node* p2) {
    Node* result = NULL;
    Node* t1 = p1;
    Node* t2 = p2;

    while (t1 != NULL && t2 != NULL) {
        if (t1->exp > t2->exp) {
            insertTerm(&result, t1->coeff, t1->exp);
            t1 = t1->next;
        } else if (t1->exp < t2->exp) {
            insertTerm(&result, t2->coeff, t2->exp);
```

```
            t2 = t2->next;
        } else {
            insertTerm(&result, t1->coeff + t2->coeff, t1->exp);
            t1 = t1->next;
            t2 = t2->next;
        }
    }

    while (t1 != NULL) {
        insertTerm(&result, t1->coeff, t1->exp);
        t1 = t1->next;
    }
    while (t2 != NULL) {
        insertTerm(&result, t2->coeff, t2->exp);
        t2 = t2->next;
    }

    return result;
}

Node* subtractPolynomials(Node* p1, Node* p2) {
    Node* result = NULL;
    Node* t1 = p1;
    Node* t2 = p2;

    while (t1 != NULL && t2 != NULL) {
        if (t1->exp > t2->exp) {
            insertTerm(&result, t1->coeff, t1->exp);
            t1 = t1->next;
        } else if (t1->exp < t2->exp) {
            insertTerm(&result, -t2->coeff, t2->exp);
            t2 = t2->next;
        } else {
            insertTerm(&result, t1->coeff - t2->coeff, t1->exp);
            t1 = t1->next;
            t2 = t2->next;
        }
    }

    while (t1 != NULL) {
        insertTerm(&result, t1->coeff, t1->exp);
        t1 = t1->next;
    }
    while (t2 != NULL) {
        insertTerm(&result, -t2->coeff, t2->exp);
        t2 = t2->next;
    }

    return result;
}

Node* multiplyPolynomials(Node* p1, Node* p2) {
    Node* result = NULL;

    for (Node* t1 = p1; t1 != NULL; t1 = t1->next) {
        for (Node* t2 = p2; t2 != NULL; t2 = t2->next) {
            int targetCoeff = t1->coeff * t2->coeff;
            int targetExp = t1->exp + t2->exp;
            insertTerm(&result, targetCoeff, targetExp);
        }
    }
    return result;
}
```

```c
void inputPolynomial(Node** headRef) {
    clearPolynomial(headRef);
    int terms, coeff, exp;

    printf("Enter the number of terms: ");
    scanf("%d", &terms);

    for (int i = 0; i < terms; i++) {
        printf("  Enter coefficient and exponent for term %d (space separated): ", i + 1);
        scanf("%d %d", &coeff, &exp);
        insertTerm(headRef, coeff, exp);
    }
}

int main() {
    Node* p1 = NULL;
    Node* p2 = NULL;
    Node* result = NULL;
    int choice;

    do {
        printf("\n================================");
        printf("\n   POLYNOMIAL OPERATIONS MENU");
        printf("\n================================");
        printf("\n1. Input Polynomials (P1 and P2)");
        printf("\n2. Display Current Polynomials");
        printf("\n3. Add (P1 + P2)");
        printf("\n4. Subtract (P1 - P2)");
        printf("\n5. Multiply (P1 * P2)");
        printf("\n6. Exit");
        printf("\nEnter your choice (1-6): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("\n--- Inputting Polynomial 1 ---\n");
                inputPolynomial(&p1);
                printf("\n--- Inputting Polynomial 2 ---\n");
                inputPolynomial(&p2);
                printf("\nPolynomials stored successfully!\n");
                break;

            case 2:
                printf("\nP1: "); displayPolynomial(p1);
                printf("P2: "); displayPolynomial(p2);
                break;

            case 3:
                clearPolynomial(&result);
                result = addPolynomials(p1, p2);
                printf("\nResult (P1 + P2): ");
                displayPolynomial(result);
                break;

            case 4:
                clearPolynomial(&result);
                result = subtractPolynomials(p1, p2);
                printf("\nResult (P1 - P2): ");
                displayPolynomial(result);
                break;

            case 5:
```

```c
                clearPolynomial(&result);
                result = multiplyPolynomials(p1, p2);
                printf("\nResult (P1 * P2): ");
                displayPolynomial(result);
                break;

            case 6:
                printf("\nExiting program. Freeing memory...\n");
                break;

            default:
                printf("\nInvalid choice! Please select an option between 1 and 6.\n");
        }
    } while (choice != 6);

    clearPolynomial(&p1);
    clearPolynomial(&p2);
    clearPolynomial(&result);

    return 0;
}
```

Output

```
$ gcc ass-2-2.c
$ ./a.out

================================
   POLYNOMIAL OPERATIONS MENU
================================
1. Input Polynomials (P1 and P2)
2. Display Current Polynomials
3. Add (P1 + P2)
4. Subtract (P1 - P2)
5. Multiply (P1 * P2)
6. Exit
Enter your choice (1-6): 1

--- Inputting Polynomial 1 ---
Enter the number of terms: 2
  Enter coefficient and exponent for term 1 (space separated): 3 2
  Enter coefficient and exponent for term 2 (space separated): 2 1

--- Inputting Polynomial 2 ---
Enter the number of terms: 2
  Enter coefficient and exponent for term 1 (space separated): 1 3
  Enter coefficient and exponent for term 2 (space separated): -4 1

Polynomials stored successfully!

================================
   POLYNOMIAL OPERATIONS MENU
================================
1. Input Polynomials (P1 and P2)
2. Display Current Polynomials
3. Add (P1 + P2)
4. Subtract (P1 - P2)
5. Multiply (P1 * P2)
6. Exit
Enter your choice (1-6): 2

P1: 3x^2 + 2x
P2: x^3 - 4x

================================
   POLYNOMIAL OPERATIONS MENU
================================
1. Input Polynomials (P1 and P2)
```

```
2. Display Current Polynomials
3. Add (P1 + P2)
4. Subtract (P1 - P2)
5. Multiply (P1 * P2)
6. Exit
Enter your choice (1-6): 3

Result (P1 + P2): x^3 + 3x^2 - 2x

================================
   POLYNOMIAL OPERATIONS MENU
================================
1. Input Polynomials (P1 and P2)
2. Display Current Polynomials
3. Add (P1 + P2)
4. Subtract (P1 - P2)
5. Multiply (P1 * P2)
6. Exit
Enter your choice (1-6): 4

Result (P1 - P2): -x^3 + 3x^2 + 6x

================================
   POLYNOMIAL OPERATIONS MENU
================================
1. Input Polynomials (P1 and P2)
2. Display Current Polynomials
3. Add (P1 + P2)
4. Subtract (P1 - P2)
5. Multiply (P1 * P2)
6. Exit
Enter your choice (1-6): 5

Result (P1 * P2): 3x^5 + 2x^4 - 12x^3 - 8x^2

================================
   POLYNOMIAL OPERATIONS MENU
================================
1. Input Polynomials (P1 and P2)
2. Display Current Polynomials
3. Add (P1 + P2)
4. Subtract (P1 - P2)
5. Multiply (P1 * P2)
6. Exit
Enter your choice (1-6): 6

Exiting program. Freeing memory...
```

—

Assignment 2 : Problem 3

Problem Statement:
Implement Doubly Linked List for the operations (I to XV) of Sl. No. 1.

ass-2-3.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

Node* createNode(int data) {
    Node* temp = (Node*)malloc(sizeof(Node));
    if (temp == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }
    temp->data = data;
    temp->prev = NULL;
    temp->next = NULL;
    return temp;
}

void printList(Node* head) {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    Node* temp = head;
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void insertFront(Node** head, int val) {
    Node* newNode = createNode(val);
    newNode->next = *head;

    if (*head != NULL) {
        (*head)->prev = newNode;
    }
    *head = newNode;
}

void insertEnd(Node** head, int val) {
    Node* newNode = createNode(val);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    Node* temp = *head;
    while (temp->next != NULL) {
```

```
        temp = temp->next;
    }

    temp->next = newNode;
    newNode->prev = temp;
}

void createList(Node** head) {
    int n, val;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter value: ");
        scanf("%d", &val);
        insertEnd(head, val);
    }
}

void insertAfterK(Node** head, int k, int val) {
    Node* temp = *head;

    for (int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) {
        printf("Invalid position.\n");
        return;
    }

    Node* newNode = createNode(val);
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
}

void insertAfterValue(Node** head, int key, int val) {
    Node* temp = *head;

    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL) {
        printf("Value not found.\n");
        return;
    }

    Node* newNode = createNode(val);
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
}

void insertBeforeK(Node** head, int k, int val) {
```

```
    if (k == 1) {
        insertFront(head, val);
        return;
    }

    Node* temp = *head;
    for (int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) {
        printf("Invalid position.\n");
        return;
    }

    Node* newNode = createNode(val);
    newNode->next = temp;
    newNode->prev = temp->prev;
    temp->prev->next = newNode;
    temp->prev = newNode;
}

void insertBeforeValue(Node** head, int key, int val) {
    if (*head == NULL) return;

    if ((*head)->data == key) {
        insertFront(head, val);
        return;
    }

    Node* temp = *head;
    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL) {
        printf("Value not found.\n");
        return;
    }

    Node* newNode = createNode(val);
    newNode->next = temp;
    newNode->prev = temp->prev;

    temp->prev->next = newNode;
    temp->prev = newNode;
}

void deleteFirst(Node** head) {
    if (*head == NULL) return;

    Node* temp = *head;
    *head = (*head)->next;

    if (*head != NULL) {
        (*head)->prev = NULL;
    }
    free(temp);
}

void deleteLast(Node** head) {
    if (*head == NULL) return;

    if ((*head)->next == NULL) {
        free(*head);
```

```
        *head = NULL;
        return;
    }

    Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->prev->next = NULL;
    free(temp);
}

void deleteAfterK(Node** head, int k) {
    Node* temp = *head;

    for (int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL || temp->next == NULL) {
        printf("Deletion not possible.\n");
        return;
    }

    Node* del = temp->next;
    temp->next = del->next;

    if (del->next != NULL) {
        del->next->prev = temp;
    }
    free(del);
}

void deleteBeforeK(Node** head, int k) {
    if (k <= 1 || *head == NULL) {
        printf("Deletion not possible.\n");
        return;
    }

    Node* temp = *head;
    for (int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL || temp->prev == NULL) {
        printf("Deletion not possible.\n");
        return;
    }

    Node* del = temp->prev;

    if (del->prev == NULL) {
        *head = temp;
        temp->prev = NULL;
    } else {
        del->prev->next = temp;
        temp->prev = del->prev;
    }
    free(del);
}

void deleteK(Node** head, int k) {
    if (*head == NULL) return;
```

```
    if (k == 1) {
        deleteFirst(head);
        return;
    }

    Node* temp = *head;
    for (int i = 1; i < k && temp != NULL; i++)
        temp = temp->next;

    if (temp == NULL) {
        printf("Invalid position.\n");
        return;
    }

    temp->prev->next = temp->next;
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    free(temp);
}

void deleteValue(Node** head, int key) {
    if (*head == NULL) return;

    if ((*head)->data == key) {
        deleteFirst(head);
        return;
    }

    Node* temp = *head;
    while (temp != NULL && temp->data != key)
        temp = temp->next;

    if (temp == NULL) {
        printf("Value not found.\n");
        return;
    }

    temp->prev->next = temp->next;
    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }
    free(temp);
}

void reverseList(Node** head) {
    if (*head == NULL) return;

    Node* current = *head;
    Node* temp = NULL;

    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;

        current = current->prev;
    }

    if (temp != NULL) {
        *head = temp->prev;
    }
}
```

```c
int main() {
    Node* list1 = NULL;
    int choice, val, k, key;

    while (1) {
        printf("\n--- MENU (Doubly Linked List) ---\n");
        printf("1. Create List 1\n");
        printf("2. Print List 1\n");
        printf("3. Insert Front\n");
        printf("4. Insert End\n");
        printf("5. Insert After kth Node\n");
        printf("6. Insert After Value\n");
        printf("7. Insert Before kth Node\n");
        printf("8. Insert Before Value\n");
        printf("9. Delete First\n");
        printf("10. Delete Last\n");
        printf("11. Delete After kth Node\n");
        printf("12. Delete Before kth Node\n");
        printf("13. Delete kth Node\n");
        printf("14. Delete By Value\n");
        printf("15. Reverse List\n");
        printf("0. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
        case 1:
            createList(&list1);
            break;
        case 2:
            printList(list1);
            break;
        case 3:
            printf("Enter value: ");
            scanf("%d", &val);
            insertFront(&list1, val);
            break;
        case 4:
            printf("Enter value: ");
            scanf("%d", &val);
            insertEnd(&list1, val);
            break;
        case 5:
            printf("Enter k and value: ");
            scanf("%d%d", &k, &val);
            insertAfterK(&list1, k, val);
            break;
        case 6:
            printf("Enter key and value: ");
            scanf("%d%d", &key, &val);
            insertAfterValue(&list1, key, val);
            break;
        case 7:
            printf("Enter k and value: ");
            scanf("%d%d", &k, &val);
            insertBeforeK(&list1, k, val);
            break;
        case 8:
            printf("Enter key and value: ");
            scanf("%d%d", &key, &val);
            insertBeforeValue(&list1, key, val);
```

```
            break;
        case 9:
            deleteFirst(&list1);
            break;
        case 10:
            deleteLast(&list1);
            break;
        case 11:
            printf("Enter k: ");
            scanf("%d", &k);
            deleteAfterK(&list1, k);
            break;
        case 12:
            printf("Enter k: ");
            scanf("%d", &k);
            deleteBeforeK(&list1, k);
            break;
        case 13:
            printf("Enter k: ");
            scanf("%d", &k);
            deleteK(&list1, k);
            break;
        case 14:
            printf("Enter value: ");
            scanf("%d", &key);
            deleteValue(&list1, key);
            break;
        case 15:
            reverseList(&list1);
            printf("List reversed.\n");
            break;
        case 0:
            while(list1 != NULL) {
                deleteFirst(&list1);
            }
            exit(0);
        default:
            printf("Invalid choice.\n");
        }
    }
    return 0;
}
```

Output

```
$ gcc ass-2-3.c
$ ./a.out

--- MENU (Doubly Linked List) ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
```

```
0. Exit
Enter choice: 1
Enter number of nodes: 5
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Enter value: 50

--- MENU (Doubly Linked List) ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
0. Exit
Enter choice: 2
10 <-> 20 <-> 30 <-> 40 <-> 50 <-> NULL

--- MENU (Doubly Linked List) ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
0. Exit
Enter choice: 3
Enter value: 99

--- MENU (Doubly Linked List) ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
0. Exit
Enter choice: 2
99 <-> 10 <-> 20 <-> 30 <-> 40 <-> 50 <-> NULL

--- MENU (Doubly Linked List) ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
```

```
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
0. Exit
Enter choice: 15
List reversed.

--- MENU (Doubly Linked List) ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
0. Exit
Enter choice: 2
50 <-> 40 <-> 30 <-> 20 <-> 10 <-> 99 <-> NULL

--- MENU (Doubly Linked List) ---
1. Create List 1
2. Print List 1
3. Insert Front
4. Insert End
5. Insert After kth Node
6. Insert After Value
7. Insert Before kth Node
8. Insert Before Value
9. Delete First
10. Delete Last
11. Delete After kth Node
12. Delete Before kth Node
13. Delete kth Node
14. Delete By Value
15. Reverse List
0. Exit
Enter choice: 0
```

—

Assignment 2 : Problem 4

Problem Statement:
Given a problem, “people are standing in a circle waiting to be executed. Counting begins at a specified point in the circle and proceeds in a specified direction (e.g., clockwise). After a specified number of people is skipped, the next person is executed (i.e., removed). The procedure is repeated with the remaining people, starting with the next person, going in the same direction and skipping the same number of people until only one person remains and is freed”. The solution is to find the person to be freed. Solve it using

i) 2D Array
ii) A circular linked list
iii) A circular doubly linked list

ass-2-4.c

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int id;
    struct Node* next;
} Node;

typedef struct DNode {
    int id;
    struct DNode* prev;
    struct DNode* next;
} DNode;

int D2Array(int n, int k) {
    int** arr = (int**)malloc(2 * sizeof(int*));
    arr[0] = (int*)malloc(n * sizeof(int));
    arr[1] = (int*)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++) {
        arr[0][i] = i + 1;
        arr[1][i] = 1;
    }

    int remaining = n;
    int index = 0;

    while (remaining > 1) {
        int count = 0;
        while (count < k) {
            if (arr[1][index] == 1) {
                count++;
                if (count == k) {
                    arr[1][index] = 0;
                    remaining--;
                }
            }
            if (remaining == 1) break;
            index = (index + 1) % n;
        }
    }

    int survivor = -1;
    for (int i = 0; i < n; i++) {
        if (arr[1][i] == 1) {
            survivor = arr[0][i];
            break;
```

```
        }
    }

    free(arr[0]);
    free(arr[1]);
    free(arr[2]);
    free(arr);

    return survivor;
}


int CircularList(int n, int k) {
    if (n == 1) return 1;

    Node* head = (Node*)malloc(sizeof(Node));
    head->id = 1;
    head->next = head;
    Node* tail = head;

    for (int i = 2; i <= n; i++) {
        Node* newNode = (Node*)malloc(sizeof(Node));
        newNode->id = i;
        newNode->next = head;
        tail->next = newNode;
        tail = newNode;
    }

    Node* curr = head;
    Node* prev = tail;

    while (curr->next != curr) {
        for (int count = 1; count < k; count++) {
            prev = curr;
            curr = curr->next;
        }

        prev->next = curr->next;
        Node* temp = curr;
        curr = curr->next;
        free(temp);
    }

    int survivor = curr->id;
    free(curr);

    return survivor;
}


int CircularDLList(int n, int k) {
    if (n == 1) return 1;

    DNode* head = (DNode*)malloc(sizeof(DNode));
    head->id = 1;
    head->next = head;
    head->prev = head;
    DNode* tail = head;

    for (int i = 2; i <= n; i++) {
        DNode* newNode = (DNode*)malloc(sizeof(DNode));
        newNode->id = i;
```

```
        newNode->next = head;
        newNode->prev = tail;
        tail->next = newNode;
        head->prev = newNode;

        tail = newNode;
    }

    DNode* curr = head;

    while (curr->next != curr) {
        for (int count = 1; count < k; count++) {
            curr = curr->next;
        }

        DNode* temp = curr;
        curr->prev->next = curr->next;
        curr->next->prev = curr->prev;

        curr = curr->next;
        free(temp);
    }

    int survivor = curr->id;
    free(curr);

    return survivor;
}

int main() {
    int choice, n, k;

    while (1) {
        printf("\nPick method:");
        printf("\n1. Solve using 2D Array");
        printf("\n2. Solve using Circular Singly Linked List");
        printf("\n3. Solve using Circular Doubly Linked List");
        printf("\n0. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);

        if (choice == 0) break;

        if (choice >= 1 && choice <= 3) {
            printf("Enter total number of people (n): ");
            scanf("%d", &n);
            printf("Enter elimination step size (k): ");
            scanf("%d", &k);

            if (n <= 0 || k <= 0) {
                printf("Invalid configuration parameters. Try values > 0.\n");
                continue;
            }
        }

        switch (choice) {
            case 1:
                printf("\n[2D Array] The survivor is person #%d\n", D2Array(n, k));
                break;
            case 2:
                printf("\n[Circular SLL] The survivor is person #%d\n", CircularList(n, k));
                break;
            case 3:
```

```
                printf("\n[Circular DLL] The survivor is person #%d\n", CircularDLList(n, k));
                break;
            default:
                printf("\nInvalid selection. Choose an option from 0 to 3.\n");
        }
    }
    return 0;
}
```

Output

```
$ gcc ass-2-4.c
$ ./a.out

Pick method:
1. Solve using 2D Array
2. Solve using Circular Singly Linked List
3. Solve using Circular Doubly Linked List
0. Exit
Enter choice: 1
Enter total number of people (n): 7
Enter elimination step size (k): 3

[2D Array] The survivor is person #4

Pick method:
1. Solve using 2D Array
2. Solve using Circular Singly Linked List
3. Solve using Circular Doubly Linked List
0. Exit
Enter choice: 2
Enter total number of people (n): 7
Enter elimination step size (k): 3

[Circular SLL] The survivor is person #4

Pick method:
1. Solve using 2D Array
2. Solve using Circular Singly Linked List
3. Solve using Circular Doubly Linked List
0. Exit
Enter choice: 3
Enter total number of people (n): 7
Enter elimination step size (k): 3

[Circular DLL] The survivor is person #4

Pick method:
1. Solve using 2D Array
2. Solve using Circular Singly Linked List
3. Solve using Circular Doubly Linked List
0. Exit
Enter choice: 0
```

—

Assignment 3 : Problem 1

Problem Statement:
Write a program to implement a stack data structure using an array.

ass-3-1.c

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

int stack[MAX];
int top = -1;

int isFull() {
    return (top == MAX - 1);
}

int isEmpty() {
    return (top == -1);
}

int peek(){
    return stack[top];
}

void push(int val){
    if (isFull()){
        printf("Stack Overflow\n");
    }else {
        stack[++top] = val;
    }
}

int pop() {
    if (isEmpty()){
        printf("Stack Underflow\n");
        return -1;
    }
    return stack[top--];
}

void display(){
    if (isEmpty()){
        printf("Stack Empty\n");
    }else {
        for (int i = top; i >= 0; i--){
            printf("%d ", stack[i]);
        }
        printf("\n");
    }
}

int main(){
    int choice, value;

    while (1) {
        printf("\n*** STACK OPERATIONS MENU ***\n");
        printf("1. Push (Insert)\n");
        printf("2. Pop (Delete)\n");
        printf("3. Peek (Top Element)\n");
        printf("4. Display Stack\n");
```

```
        printf("5. Exit\n");
        printf("Enter your choice (1-5): ");

        if (scanf("%d", &choice) != 1) {
            printf("Invalid input. Please enter a number.\n");
            while (getchar() != '\n');
            continue;
        }

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                push(value);
                break;
            case 2:

                printf("Popped element: %d\n", pop());
                break;

            case 3:
                if (isEmpty()) {
                    printf("Stack is empty.\n");
                } else {
                    printf("Top element is: %d\n", peek());
                }
                break;

            case 4:
                display();
                break;

            case 5:
                printf("Exiting program. Goodbye!\n");
                exit(0);

            default:
                printf("Invalid choice! Please select between 1 and 5.\n");
        }
    }
    return 0;
}
```

Output

```
$ gcc ass-3-1.c
$ ./a.out

*** STACK OPERATIONS MENU ***
1. Push (Insert)
2. Pop (Delete)
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice (1-5): 1
Enter value to push: 10

*** STACK OPERATIONS MENU ***
1. Push (Insert)
2. Pop (Delete)
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice (1-5): 1
```

```
Enter value to push: 20

*** STACK OPERATIONS MENU ***
1. Push (Insert)
2. Pop (Delete)
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice (1-5): 1
Enter value to push: 30

*** STACK OPERATIONS MENU ***
1. Push (Insert)
2. Pop (Delete)
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice (1-5): 4
30 20 10

*** STACK OPERATIONS MENU ***
1. Push (Insert)
2. Pop (Delete)
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice (1-5): 3
Top element is: 30

*** STACK OPERATIONS MENU ***
1. Push (Insert)
2. Pop (Delete)
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice (1-5): 2
Popped element: 30

*** STACK OPERATIONS MENU ***
1. Push (Insert)
2. Pop (Delete)
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice (1-5): 4
20 10

*** STACK OPERATIONS MENU ***
1. Push (Insert)
2. Pop (Delete)
3. Peek (Top Element)
4. Display Stack
5. Exit
Enter your choice (1-5): 5
Exiting program. Goodbye!
```

—

Assignment 3 : Problem 2

Problem Statement:
Implement a system that can handle more than one stack (n stacks).

ass-3-2.c

```c
#include <stdio.h>

#define K 3
#define SIZE 100

int arr[SIZE];
int top[K];
int next[SIZE];
int freeTop;

void initialize() {
    int i;

    for(i=0;i<K;i++)
        top[i] = -1;

    for(i=0;i<SIZE-1;i++)
        next[i] = i + 1;

    next[SIZE-1] = -1;
    freeTop = 0;
}

void push(int item, int sn) {
    if(freeTop == -1) {
        printf("Stack Overflow\n");
        return;
    }

    int i = freeTop;
    freeTop = next[i];

    arr[i] = item;
    next[i] = top[sn];
    top[sn] = i;
}

int pop(int sn) {
    if(top[sn] == -1) {
        printf("Stack Underflow\n");
        return -1;
    }

    int i = top[sn];
    top[sn] = next[i];

    next[i] = freeTop;
    freeTop = i;

    return arr[i];
}

void display(int sn) {
    int i = top[sn];
```

```
    if(i == -1) {
        printf("Stack Empty\n");
        return;
    }

    while(i != -1) {
        printf("%d ", arr[i]);
        i = next[i];
    }
    printf("\n");
}

int main() {
    int choice, stackNo, item;

    initialize();

    do {
        printf("\n1.Push\n2.Pop\n3.Display\n4.Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter Stack Number(0-2): ");
                scanf("%d", &stackNo);
                printf("Enter Item: ");
                scanf("%d", &item);
                push(item, stackNo);
                break;

            case 2:
                printf("Enter Stack Number(0-2): ");
                scanf("%d", &stackNo);
                printf("Deleted = %d\n", pop(stackNo));
                break;

            case 3:
                printf("Enter Stack Number(0-2): ");
                scanf("%d", &stackNo);
                display(stackNo);
                break;
        }

    } while(choice != 4);

    return 0;
}
```

Output

```
$ gcc ass-3-2.c
$ ./a.out

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 1
Enter Stack Number(0-2): 0
Enter Item: 10

1.Push
```

```
2.Pop
3.Display
4.Exit
Enter your choice: 1
Enter Stack Number(0-2): 1
Enter Item: 20

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 1
Enter Stack Number(0-2): 0
Enter Item: 15

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 3
Enter Stack Number(0-2): 0
15 10

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 3
Enter Stack Number(0-2): 1
20

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 2
Enter Stack Number(0-2): 0
Deleted = 15

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 4
```

—

Assignment 3 : Problem 3

Problem Statement:
Print the data from a file of integers in reverse order using a stack.

ass-3-3.c

```c
#include <stdio.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int x) {
    stack[++top] = x;
}

int pop() {
    return stack[top--];
}

int main() {

    FILE *fp;
    int num;
    char filename[50];

    printf("Enter File Name: ");
    scanf("%s", filename);

    fp = fopen(filename, "r");

    if(fp == NULL) {
        printf("File Not Found\n");
        return 0;
    }

    while(fscanf(fp, "%d", &num) != EOF)
        push(num);

    fclose(fp);

    printf("Reverse Order:\n");

    while(top != -1)
        printf("%d ", pop());
    printf("\n");

    return 0;
}
```

Output

```
$ gcc ass-3-3.c
$ ./a.out
Enter File Name: nums.txt
Reverse Order:
17 3 21 8 12 5
```

—

Assignment 3 : Problem 4

Problem Statement:
Write a Boolean function to return true if two stacks are equal.

ass-3-4.c

```c
#include <stdio.h>

int main() {

    int n1, n2, i;
    int s1[100], s2[100];
    int equal = 1;

    printf("Enter Size of Stack1: ");
    scanf("%d", &n1);

    printf("Enter Elements:\n");
    for(i=0;i<n1;i++)
        scanf("%d",&s1[i]);

    printf("Enter Size of Stack2: ");
    scanf("%d",&n2);

    printf("Enter Elements:\n");
    for(i=0;i<n2;i++)
        scanf("%d",&s2[i]);

    if(n1 != n2)
        equal = 0;
    else {
        for(i=0;i<n1;i++) {
            if(s1[i] != s2[i]) {
                equal = 0;
                break;
            }
        }
    }

    if(equal)
        printf("Stacks are Equal\n");
    else
        printf("Stacks are Not Equal\n");

    return 0;
}
```

Output

```
$ gcc ass-3-4.c
$ ./a.out
Enter Size of Stack1: 3
Enter Elements:
1 2 3
Enter Size of Stack2: 3
Enter Elements:
1 2 3
Stacks are Equal
```

—

Assignment 3 : Problem 5

Problem Statement:
Write a program for dynamic implementation (using a link list) of stacks (n stacks).

ass-3-5.c

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *top = NULL;

void push(int x) {
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = x;
    newNode->next = top;
    top = newNode;
}

int pop() {
    if(top == NULL) {
        printf("Stack Empty\n");
        return -1;
    }

    struct Node *temp = top;
    int val = temp->data;

    top = top->next;
    free(temp);

    return val;
}

void display() {
    struct Node *temp = top;

    while(temp) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {

    int choice, item;

    do {
        printf("\n1.Push\n2.Pop\n3.Display\n4.Exit\n");
        printf("Enter your choice: ");
        scanf("%d",&choice);

        switch(choice) {
```

```
            case 1:
                printf("Enter Item: ");
                scanf("%d",&item);
                push(item);
                break;

            case 2:
                printf("Deleted = %d\n", pop());
                break;

            case 3:
                display();
                break;
        }

    } while(choice != 4);
    printf("Exiting program.\n");

    return 0;
}
```

Output

```
$ gcc ass-3-5.c
$ ./a.out

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 1
Enter Item: 10

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 1
Enter Item: 20

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 1
Enter Item: 30

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 3
30 -> 20 -> 10 -> NULL

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 2
Deleted = 30

1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 3
20 -> 10 -> NULL
```

```
1.Push
2.Pop
3.Display
4.Exit
Enter your choice: 4
Exiting program.
```

—

Assignment 3 : Problem 6

Problem Statement:
Using a stack, write a program to convert an infix expression into its equivalent postfix expression.

ass-3-6.c

```c
#include <stdio.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char ch)
{
    stack[++top] = ch;
}

char pop()
{
    if(top == -1)
        return '\0';

    return stack[top--];
}

char peek()
{
    if(top == -1)
        return '\0';

    return stack[top];
}

int precedence(char op)
{
    switch(op)
    {
        case '^':
            return 3;

        case '*':
        case '/':
            return 2;

        case '+':
        case '-':
            return 1;

        default:
            return 0;
    }
}

int main()
{
    char infix[MAX];
    char postfix[MAX];
    int i, j = 0;
```

```
    printf("Enter Infix Expression: ");
    scanf("%s", infix);

    for(i = 0; infix[i] != '\0'; i++)
    {
        char ch = infix[i];

        if(isalnum(ch))
        {
            postfix[j++] = ch;
        }

        else if(ch == '(')
        {
            push(ch);
        }

        else if(ch == ')')
        {
            while(top != -1 && peek() != '(')
            {
                postfix[j++] = pop();
            }

            pop();
        }

        else
        {
            while(top != -1 && precedence(peek()) >= precedence(ch))
            {
                postfix[j++] = pop();
            }

            push(ch);
        }
    }

    while(top != -1)
    {
        postfix[j++] = pop();
    }

    postfix[j] = '\0';

    printf("Postfix Expression = %s\n", postfix);

    return 0;
}
```

Output

```
$ gcc ass-3-6.c
$ ./a.out
Enter Infix Expression: (A+B)*(C-D)/E
Postfix Expression = AB+CD-*E/
```

—

Assignment 3 : Problem 7

Problem Statement:
Write a program to evaluate postfix expression using a stack.

ass-3-7.c

```c
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <stdlib.h>

#define MAX 100

int stack[MAX];
int top = -1;

void push(int x)
{
    if (top == MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = x;
}

int pop()
{
    if(top == -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }

    return stack[top--];
}

int main()
{
    char postfix[MAX];
    int op1, op2, result;

    printf("Enter Space-Separated Postfix Expression: ");
    fgets(postfix, MAX, stdin);

    char *token = strtok(postfix, " \n");

    while (token != NULL)
    {
        if (strlen(token) == 1 && (token[0] == '+' || token[0] == '-' ||
                                   token[0] == '*' || token[0] == '/' || token[0] == '^'))
        {
            char ch = token[0];
            op2 = pop();
            op1 = pop();

            switch(ch)
            {
                case '+':
                    result = op1 + op2;
                    break;
```

```
                case '-':
                    result = op1 - op2;
                    break;

                case '*':
                    result = op1 * op2;
                    break;

                case '/':
                    if (op2 == 0) {
                        printf("Error: Division by zero\n");
                        return 1;
                    }
                    result = op1 / op2;
                    break;

                case '^':
                    result = (int)pow(op1, op2);
                    break;
            }

            push(result);
        }
        else
        {
            push(atoi(token));
        }
\
        token = strtok(NULL, " \n");
    }

    printf("Result = %d\n", pop());

    return 0;
}
```

Output

```
$ gcc ass-3-7.c
$ ./a.out
Enter Space-Separated Postfix Expression: 5 3 + 2 *
Result = 16
```

—

Assignment 3 : Problem 8

Problem Statement:
Write a program to check balanced brackets of an expression using stack.

ass-3-8.c

```c
#include <stdio.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char ch)
{
    stack[++top] = ch;
}

char pop()
{
    if(top == -1)
        return '\0';

    return stack[top--];
}

int match(char open, char close)
{
    if(open == '(' && close == ')')
        return 1;

    if(open == '{' && close == '}')
        return 1;

    if(open == '[' && close == ']')
        return 1;

    return 0;
}

int main()
{
    char exp[MAX];
    int i;
    char ch, temp;

    printf("Enter Expression: ");
    scanf("%s", exp);

    for(i = 0; exp[i] != '\0'; i++)
    {
        ch = exp[i];

        if(ch == '(' || ch == '{' || ch == '[')
        {
            push(ch);
        }

        else if(ch == ')' || ch == '}' || ch == ']')
        {
            if(top == -1)
```

```
            {
                printf("Not Balanced\n");
                return 0;
            }

            temp = pop();

            if(!match(temp, ch))
            {
                printf("Not Balanced\n");
                return 0;
            }
        }
    }

    if(top == -1)
        printf("Balanced Expression\n");
    else
        printf("Not Balanced\n");

    return 0;
}
```

Output

```
$ gcc ass-3-8.c
$ ./a.out
Enter Expression: {[(A+B)*(C-D)]/E}
Balanced Expression
```

—

Assignment 3 : Problem 9

Problem Statement:
Write a program to implement queue data structure using an array.

ass-3-9.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int queue[MAX];
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if(rear == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    if(front == -1)
        front = 0;

    queue[++rear] = value;

    printf("%d inserted successfully.\n", value);
}

void dequeue()
{
    if(front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
        return;
    }

    printf("Deleted Element = %d\n", queue[front]);

    front++;

    if(front > rear)
    {
        front = -1;
        rear = -1;
    }
}

void peek()
{
    if(front == -1)
    {
        printf("Queue is Empty\n");
        return;
    }

    printf("Front Element = %d\n", queue[front]);
}
```

```c
void display()
{
    int i;

    if(front == -1)
    {
        printf("Queue is Empty\n");
        return;
    }

    printf("Queue Elements:\n");

    for(i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }

    printf("\n");
}

int main()
{
    int choice, value;

    while(1)
    {
        printf("\n========== QUEUE MENU ==========\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");
        printf("================================\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Value: ");
                scanf("%d", &value);

                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                printf("Program Terminated.\n");
                exit(0);

            default:
```

```
                printf("Invalid Choice!\n");
            }
        }

        return 0;
}
```

Output

```
$ gcc ass-3-9.c
$ ./a.out

========== QUEUE MENU ==========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
================================
Enter Choice: 1
Enter Value: 10
10 inserted successfully.

========== QUEUE MENU ==========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
================================
Enter Choice: 1
Enter Value: 20
20 inserted successfully.

========== QUEUE MENU ==========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
================================
Enter Choice: 1
Enter Value: 30
30 inserted successfully.

========== QUEUE MENU ==========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
================================
Enter Choice: 4
Queue Elements:
10 20 30

========== QUEUE MENU ==========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
================================
Enter Choice: 3
Front Element = 10

========== QUEUE MENU ==========
1. Enqueue
2. Dequeue
3. Peek
```

```
4. Display
5. Exit
================================
Enter Choice: 2
Deleted Element = 10

========== QUEUE MENU ==========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
================================
Enter Choice: 4
Queue Elements:
20 30

========== QUEUE MENU ==========
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
================================
Enter Choice: 5
Program Terminated.
```

—

Assignment 3 : Problem 10

Problem Statement:
Implement a system that can handle multiple queues (n queues).

ass-3-10.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAXQ 5
#define SIZE 100

int queue[MAXQ][SIZE];
int front[MAXQ];
int rear[MAXQ];

void initialize()
{
    int i;

    for(i = 0; i < MAXQ; i++)
    {
        front[i] = -1;
        rear[i] = -1;
    }
}

void enqueue(int qno, int value)
{
    if(qno < 1 || qno > MAXQ)
    {
        printf("Invalid Queue Number\n");
        return;
    }

    qno--;

    if(rear[qno] == SIZE - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    if(front[qno] == -1)
        front[qno] = 0;

    queue[qno][++rear[qno]] = value;

    printf("%d inserted into Queue %d\n",
           value, qno + 1);
}

void dequeue(int qno)
{
    if(qno < 1 || qno > MAXQ)
    {
        printf("Invalid Queue Number\n");
        return;
    }

    qno--;
```

```
    if(front[qno] == -1 ||
       front[qno] > rear[qno])
    {
        printf("Queue Underflow\n");
        return;
    }

    printf("Deleted = %d\n",
           queue[qno][front[qno]]);

    front[qno]++;

    if(front[qno] > rear[qno])
    {
        front[qno] = -1;
        rear[qno] = -1;
    }
}

void display(int qno)
{
    int i;

    if(qno < 1 || qno > MAXQ)
    {
        printf("Invalid Queue Number\n");
        return;
    }

    qno--;

    if(front[qno] == -1)
    {
        printf("Queue Empty\n");
        return;
    }

    printf("Queue %d : ", qno + 1);

    for(i = front[qno];
        i <= rear[qno];
        i++)
    {
        printf("%d ",
               queue[qno][i]);
    }

    printf("\n");
}

int main()
{
    int choice;
    int qno;
    int value;

    initialize();

    while(1)
    {
        printf("\n===== MULTIPLE QUEUES =====\n");
        printf("1. Enqueue\n");
```

```
        printf("2. Dequeue\n");
        printf("3. Display Queue\n");
        printf("4. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:

                printf("Enter Queue Number (1-%d): ",
                       MAXQ);
                scanf("%d", &qno);

                printf("Enter Value: ");
                scanf("%d", &value);

                enqueue(qno, value);
                break;

            case 2:

                printf("Enter Queue Number (1-%d): ",
                       MAXQ);
                scanf("%d", &qno);

                dequeue(qno);
                break;

            case 3:

                printf("Enter Queue Number (1-%d): ",
                       MAXQ);
                scanf("%d", &qno);

                display(qno);
                break;

            case 4:
                exit(0);

            default:
                printf("Invalid Choice\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-10.c
$ ./a.out

===== MULTIPLE QUEUES =====
1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Enter Choice: 1
Enter Queue Number (1-5): 1
Enter Value: 10
```

```
10 inserted into Queue 1

===== MULTIPLE QUEUES =====
1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Enter Choice: 1
Enter Queue Number (1-5): 2
Enter Value: 20
20 inserted into Queue 2

===== MULTIPLE QUEUES =====
1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Enter Choice: 1
Enter Queue Number (1-5): 1
Enter Value: 15
15 inserted into Queue 1

===== MULTIPLE QUEUES =====
1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Enter Choice: 3
Enter Queue Number (1-5): 1
Queue 1 : 10 15

===== MULTIPLE QUEUES =====
1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Enter Choice: 3
Enter Queue Number (1-5): 2
Queue 2 : 20

===== MULTIPLE QUEUES =====
1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Enter Choice: 2
Enter Queue Number (1-5): 1
Deleted = 10

===== MULTIPLE QUEUES =====
1. Enqueue
2. Dequeue
3. Display Queue
4. Exit
Enter Choice: 4
```

—

Assignment 3 : Problem 11

Problem Statement:
Append a queue x at the end of a queue y.

ass-3-11.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int qx[MAX];
int fx = -1, rx = -1;

int qy[MAX];
int fy = -1, ry = -1;

void enqueueX(int value)
{
    if(rx == MAX - 1)
    {
        printf("Queue X Overflow\n");
        return;
    }

    if(fx == -1)
        fx = 0;

    qx[++rx] = value;
}

void enqueueY(int value)
{
    if(ry == MAX - 1)
    {
        printf("Queue Y Overflow\n");
        return;
    }

    if(fy == -1)
        fy = 0;

    qy[++ry] = value;
}

int dequeueX()
{
    int value;

    if(fx == -1)
        return -1;

    value = qx[fx++];

    if(fx > rx)
    {
        fx = -1;
        rx = -1;
    }

    return value;
```

```c
}

void displayX()
{
    int i;

    if(fx == -1)
    {
        printf("Queue X Empty\n");
        return;
    }

    printf("Queue X : ");

    for(i = fx; i <= rx; i++)
        printf("%d ", qx[i]);

    printf("\n");
}

void displayY()
{
    int i;

    if(fy == -1)
    {
        printf("Queue Y Empty\n");
        return;
    }

    printf("Queue Y : ");

    for(i = fy; i <= ry; i++)
        printf("%d ", qy[i]);

    printf("\n");
}

void appendQueue()
{
    int value;

    if(fx == -1)
    {
        printf("Queue X Empty\n");
        return;
    }

    while(fx != -1)
    {
        value = dequeueX();
        enqueueY(value);
    }

    printf("Queue X appended to Queue Y successfully.\n");
}

int main()
{
    int choice;
    int value;

    while(1)
```

```
    {
        printf("\n===== APPEND QUEUES =====\n");
        printf("1. Insert into Queue X\n");
        printf("2. Insert into Queue Y\n");
        printf("3. Display Queue X\n");
        printf("4. Display Queue Y\n");
        printf("5. Append X to Y\n");
        printf("6. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Value: ");
                scanf("%d", &value);
                enqueueX(value);
                break;

            case 2:
                printf("Enter Value: ");
                scanf("%d", &value);
                enqueueY(value);
                break;

            case 3:
                displayX();
                break;

            case 4:
                displayY();
                break;

            case 5:
                appendQueue();
                break;

            case 6:
                exit(0);

            default:
                printf("Invalid Choice\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-11.c
$ ./a.out

===== APPEND QUEUES =====
1. Insert into Queue X
2. Insert into Queue Y
3. Display Queue X
4. Display Queue Y
5. Append X to Y
6. Exit
Enter Choice: 1
Enter Value: 10
```

```
===== APPEND QUEUES =====
1. Insert into Queue X
2. Insert into Queue Y
3. Display Queue X
4. Display Queue Y
5. Append X to Y
6. Exit
Enter Choice: 1
Enter Value: 20

===== APPEND QUEUES =====
1. Insert into Queue X
2. Insert into Queue Y
3. Display Queue X
4. Display Queue Y
5. Append X to Y
6. Exit
Enter Choice: 2
Enter Value: 100

===== APPEND QUEUES =====
1. Insert into Queue X
2. Insert into Queue Y
3. Display Queue X
4. Display Queue Y
5. Append X to Y
6. Exit
Enter Choice: 3
Queue X : 10 20

===== APPEND QUEUES =====
1. Insert into Queue X
2. Insert into Queue Y
3. Display Queue X
4. Display Queue Y
5. Append X to Y
6. Exit
Enter Choice: 4
Queue Y : 100

===== APPEND QUEUES =====
1. Insert into Queue X
2. Insert into Queue Y
3. Display Queue X
4. Display Queue Y
5. Append X to Y
6. Exit
Enter Choice: 5
Queue X appended to Queue Y successfully.

===== APPEND QUEUES =====
1. Insert into Queue X
2. Insert into Queue Y
3. Display Queue X
4. Display Queue Y
5. Append X to Y
6. Exit
Enter Choice: 4
Queue Y : 100 10 20

===== APPEND QUEUES =====
1. Insert into Queue X
2. Insert into Queue Y
3. Display Queue X
4. Display Queue Y
5. Append X to Y
6. Exit
Enter Choice: 6
```

—

Assignment 3 : Problem 12

Problem Statement:
A Boolean function to return true if two eq(q1, q2) are equal.

ass-3-12.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int q1[MAX];
int f1 = -1, r1 = -1;

int q2[MAX];
int f2 = -1, r2 = -1;

void enqueueQ1(int value)
{
    if(r1 == MAX - 1)
    {
        printf("Queue 1 Overflow\n");
        return;
    }

    if(f1 == -1)
        f1 = 0;

    q1[++r1] = value;
}

void enqueueQ2(int value)
{
    if(r2 == MAX - 1)
    {
        printf("Queue 2 Overflow\n");
        return;
    }

    if(f2 == -1)
        f2 = 0;

    q2[++r2] = value;
}

void displayQ1()
{
    int i;

    if(f1 == -1)
    {
        printf("Queue 1 Empty\n");
        return;
    }

    printf("Queue 1 : ");

    for(i = f1; i <= r1; i++)
        printf("%d ", q1[i]);

    printf("\n");
```

```c
}

void displayQ2()
{
    int i;

    if(f2 == -1)
    {
        printf("Queue 2 Empty\n");
        return;
    }

    printf("Queue 2 : ");

    for(i = f2; i <= r2; i++)
        printf("%d ", q2[i]);

    printf("\n");
}

int eq()
{
    int i, j;

    int size1, size2;

    if(f1 == -1 && f2 == -1)
        return 1;

    if(f1 == -1 || f2 == -1)
        return 0;

    size1 = r1 - f1 + 1;
    size2 = r2 - f2 + 1;

    if(size1 != size2)
        return 0;

    i = f1;
    j = f2;

    while(i <= r1 && j <= r2)
    {
        if(q1[i] != q2[j])
            return 0;

        i++;
        j++;
    }

    return 1;
}

int main()
{
    int choice, value;

    while(1)
    {
        printf("\n===== QUEUE COMPARISON =====\n");
        printf("1. Insert into Queue 1\n");
        printf("2. Insert into Queue 2\n");
        printf("3. Display Queue 1\n");
```

```
        printf("4. Display Queue 2\n");
        printf("5. Compare Queues\n");
        printf("6. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Value: ");
                scanf("%d", &value);
                enqueueQ1(value);
                break;

            case 2:
                printf("Enter Value: ");
                scanf("%d", &value);
                enqueueQ2(value);
                break;

            case 3:
                displayQ1();
                break;

            case 4:
                displayQ2();
                break;

            case 5:
                if(eq())
                    printf("Queues are Equal\n");
                else
                    printf("Queues are NOT Equal\n");
                break;

            case 6:
                exit(0);

            default:
                printf("Invalid Choice\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-12.c
$ ./a.out

===== QUEUE COMPARISON =====
1. Insert into Queue 1
2. Insert into Queue 2
3. Display Queue 1
4. Display Queue 2
5. Compare Queues
6. Exit
Enter Choice: 1
Enter Value: 10

===== QUEUE COMPARISON =====
```

```
1. Insert into Queue 1
2. Insert into Queue 2
3. Display Queue 1
4. Display Queue 2
5. Compare Queues
6. Exit
Enter Choice: 1
Enter Value: 20

===== QUEUE COMPARISON =====
1. Insert into Queue 1
2. Insert into Queue 2
3. Display Queue 1
4. Display Queue 2
5. Compare Queues
6. Exit
Enter Choice: 2
Enter Value: 10

===== QUEUE COMPARISON =====
1. Insert into Queue 1
2. Insert into Queue 2
3. Display Queue 1
4. Display Queue 2
5. Compare Queues
6. Exit
Enter Choice: 2
Enter Value: 20

===== QUEUE COMPARISON =====
1. Insert into Queue 1
2. Insert into Queue 2
3. Display Queue 1
4. Display Queue 2
5. Compare Queues
6. Exit
Enter Choice: 5
Queues are Equal

===== QUEUE COMPARISON =====
1. Insert into Queue 1
2. Insert into Queue 2
3. Display Queue 1
4. Display Queue 2
5. Compare Queues
6. Exit
Enter Choice: 6
```

—

Assignment 3 : Problem 13

Problem Statement:
Reverse a queue.

ass-3-13.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int queue[MAX];
int front = -1;
int rear = -1;

int stack[MAX];
int top = -1;

void enqueue(int value)
{
    if(rear == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    if(front == -1)
        front = 0;

    queue[++rear] = value;
}

int dequeue()
{
    int value;

    if(front == -1)
    {
        printf("Queue Underflow\n");
        return -1;
    }

    value = queue[front++];

    if(front > rear)
    {
        front = -1;
        rear = -1;
    }

    return value;
}

void push(int value)
{
    if(top == MAX - 1)
    {
        printf("Stack Overflow\n");
        return;
    }
```

```
    stack[++top] = value;
}

int pop()
{
    if(top == -1)
    {
        printf("Stack Underflow\n");
        return -1;
    }

    return stack[top--];
}

void display()
{
    int i;

    if(front == -1)
    {
        printf("Queue Empty\n");
        return;
    }

    printf("Queue : ");

    for(i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }

    printf("\n");
}

void reverseQueue()
{
    int value;

    if(front == -1)
    {
        printf("Queue Empty\n");
        return;
    }

    while(front != -1)
    {
        value = dequeue();
        push(value);
    }

    while(top != -1)
    {
        value = pop();
        enqueue(value);
    }

    printf("Queue Reversed Successfully\n");
}

int main()
{
    int choice, value;
```

```c
    while(1)
    {
        printf("\n===== REVERSE QUEUE =====\n");
        printf("1. Enqueue\n");
        printf("2. Display Queue\n");
        printf("3. Reverse Queue\n");
        printf("4. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                display();
                break;

            case 3:
                reverseQueue();
                break;

            case 4:
                exit(0);

            default:
                printf("Invalid Choice\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-13.c
$ ./a.out

===== REVERSE QUEUE =====
1. Enqueue
2. Display Queue
3. Reverse Queue
4. Exit
Enter Choice: 1
Enter Value: 10

===== REVERSE QUEUE =====
1. Enqueue
2. Display Queue
3. Reverse Queue
4. Exit
Enter Choice: 1
Enter Value: 20

===== REVERSE QUEUE =====
1. Enqueue
2. Display Queue
3. Reverse Queue
4. Exit
Enter Choice: 1
```

```
Enter Value: 30

===== REVERSE QUEUE =====
1. Enqueue
2. Display Queue
3. Reverse Queue
4. Exit
Enter Choice: 2
Queue : 10 20 30

===== REVERSE QUEUE =====
1. Enqueue
2. Display Queue
3. Reverse Queue
4. Exit
Enter Choice: 3
Queue Reversed Successfully

===== REVERSE QUEUE =====
1. Enqueue
2. Display Queue
3. Reverse Queue
4. Exit
Enter Choice: 2
Queue : 30 20 10

===== REVERSE QUEUE =====
1. Enqueue
2. Display Queue
3. Reverse Queue
4. Exit
Enter Choice: 4
```

—

Assignment 3 : Problem 14

Problem Statement:
Procedure replace(p, e, x) to replace every occurrence of element e in queue p with the value of x.

ass-3-14.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

int queue[MAX];
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if(rear == MAX - 1)
    {
        printf("Queue Overflow\n");
        return;
    }

    if(front == -1)
        front = 0;

    queue[++rear] = value;
}

void display()
{
    int i;

    if(front == -1)
    {
        printf("Queue Empty\n");
        return;
    }

    printf("Queue : ");

    for(i = front; i <= rear; i++)
        printf("%d ", queue[i]);

    printf("\n");
}

void replaceElement(int e, int x)
{
    int i;
    int found = 0;

    if(front == -1)
    {
        printf("Queue Empty\n");
        return;
    }

    for(i = front; i <= rear; i++)
    {
        if(queue[i] == e)
```

```c
        {
            queue[i] = x;
            found = 1;
        }
    }

    if(found)
        printf("All occurrences replaced successfully.\n");
    else
        printf("Element not found in queue.\n");
}

int main()
{
    int choice;
    int value;
    int e, x;

    while(1)
    {
        printf("\n===== REPLACE IN QUEUE =====\n");
        printf("1. Enqueue\n");
        printf("2. Display Queue\n");
        printf("3. Replace Element\n");
        printf("4. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Value: ");
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                display();
                break;

            case 3:
                printf("Enter element to replace (e): ");
                scanf("%d", &e);

                printf("Enter new value (x): ");
                scanf("%d", &x);

                replaceElement(e, x);
                break;

            case 4:
                exit(0);

            default:
                printf("Invalid Choice\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-14.c
$ ./a.out

===== REPLACE IN QUEUE =====
1. Enqueue
2. Display Queue
3. Replace Element
4. Exit
Enter Choice: 1
Enter Value: 10

===== REPLACE IN QUEUE =====
1. Enqueue
2. Display Queue
3. Replace Element
4. Exit
Enter Choice: 1
Enter Value: 20

===== REPLACE IN QUEUE =====
1. Enqueue
2. Display Queue
3. Replace Element
4. Exit
Enter Choice: 1
Enter Value: 10

===== REPLACE IN QUEUE =====
1. Enqueue
2. Display Queue
3. Replace Element
4. Exit
Enter Choice: 2
Queue : 10 20 10

===== REPLACE IN QUEUE =====
1. Enqueue
2. Display Queue
3. Replace Element
4. Exit
Enter Choice: 3
Enter element to replace (e): 10
Enter new value (x): 99
All occurrences replaced successfully.

===== REPLACE IN QUEUE =====
1. Enqueue
2. Display Queue
3. Replace Element
4. Exit
Enter Choice: 2
Queue : 99 20 99

===== REPLACE IN QUEUE =====
1. Enqueue
2. Display Queue
3. Replace Element
4. Exit
Enter Choice: 4
```

—

Assignment 3 : Problem 15

Problem Statement:
Write a program for dynamic implementation (using a link list) of a queue of the above.

ass-3-15.c

```c
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *next;
};

typedef struct Node Node;

Node *front = NULL;
Node *rear = NULL;

void enqueue(int value)
{
    Node *newNode;

    newNode = (Node *)malloc(sizeof(Node));

    if(newNode == NULL)
    {
        printf("Memory Allocation Failed\n");
        return;
    }

    newNode->data = value;
    newNode->next = NULL;

    if(front == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted successfully.\n", value);
}

void dequeue()
{
    Node *temp;

    if(front == NULL)
    {
        printf("Queue Underflow\n");
        return;
    }

    temp = front;

    printf("Deleted Element = %d\n", front->data);
```

```c
    front = front->next;

    if(front == NULL)
        rear = NULL;

    free(temp);
}

void peek()
{
    if(front == NULL)
    {
        printf("Queue Empty\n");
        return;
    }

    printf("Front Element = %d\n", front->data);
}

void display()
{
    Node *temp;

    if(front == NULL)
    {
        printf("Queue Empty\n");
        return;
    }

    temp = front;

    printf("Queue : ");

    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}

int main()
{
    int choice, value;

    while(1)
    {
        printf("\n===== QUEUE USING LINKED LIST =====\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Value: ");
```

```
                scanf("%d", &value);
                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                printf("Program Terminated.\n");
                exit(0);

            default:
                printf("Invalid Choice\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-15.c
$ ./a.out

===== QUEUE USING LINKED LIST =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 1
Enter Value: 10
10 inserted successfully.

===== QUEUE USING LINKED LIST =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 1
Enter Value: 20
20 inserted successfully.

===== QUEUE USING LINKED LIST =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 1
Enter Value: 30
30 inserted successfully.

===== QUEUE USING LINKED LIST =====
1. Enqueue
2. Dequeue
3. Peek
```

```
4. Display
5. Exit
Enter Choice: 4
Queue : 10 20 30

===== QUEUE USING LINKED LIST =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 3
Front Element = 10

===== QUEUE USING LINKED LIST =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 2
Deleted Element = 10

===== QUEUE USING LINKED LIST =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 4
Queue : 20 30

===== QUEUE USING LINKED LIST =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 5
Program Terminated.
```

—

Assignment 3 : Problem 16

Problem Statement:
Implement a circular queue using an array.

ass-3-16.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

void enqueue(int value)
{
    if((front == 0 && rear == MAX - 1) ||
       (front == rear + 1))
    {
        printf("Queue Overflow\n");
        return;
    }

    if(front == -1)
    {
        front = rear = 0;
    }
    else if(rear == MAX - 1)
    {
        rear = 0;
    }
    else
    {
        rear++;
    }

    queue[rear] = value;

    printf("%d inserted successfully.\n", value);
}

void dequeue()
{
    if(front == -1)
    {
        printf("Queue Underflow\n");
        return;
    }

    printf("Deleted Element = %d\n", queue[front]);

    if(front == rear)
    {
        front = rear = -1;
    }
    else if(front == MAX - 1)
    {
        front = 0;
    }
    else
```

```c
    {
        front++;
    }
}

void peek()
{
    if(front == -1)
    {
        printf("Queue Empty\n");
        return;
    }

    printf("Front Element = %d\n", queue[front]);
}

void display()
{
    int i;

    if(front == -1)
    {
        printf("Queue Empty\n");
        return;
    }

    printf("Queue Elements : ");

    if(front <= rear)
    {
        for(i = front; i <= rear; i++)
            printf("%d ", queue[i]);
    }
    else
    {
        for(i = front; i < MAX; i++)
            printf("%d ", queue[i]);

        for(i = 0; i <= rear; i++)
            printf("%d ", queue[i]);
    }

    printf("\n");
}

int main()
{
    int choice;
    int value;

    while(1)
    {
        printf("\n===== CIRCULAR QUEUE =====\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Peek\n");
        printf("4. Display\n");
        printf("5. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
```

```c
        {
            case 1:
                printf("Enter Value: ");
                scanf("%d", &value);

                enqueue(value);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                exit(0);

            default:
                printf("Invalid Choice\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-16.c
$ ./a.out

===== CIRCULAR QUEUE =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 1
Enter Value: 10
10 inserted successfully.

===== CIRCULAR QUEUE =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 1
Enter Value: 20
20 inserted successfully.

===== CIRCULAR QUEUE =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 1
Enter Value: 30
30 inserted successfully.
```

```
===== CIRCULAR QUEUE =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 4
Queue Elements : 10 20 30

===== CIRCULAR QUEUE =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 2
Deleted Element = 10

===== CIRCULAR QUEUE =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 1
Enter Value: 40
40 inserted successfully.

===== CIRCULAR QUEUE =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 4
Queue Elements : 20 30 40

===== CIRCULAR QUEUE =====
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter Choice: 5
```

—

Assignment 3 : Problem 17

Problem Statement:
Implement a Deque using a doubly linked list.

ass-3-17.c

```c
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int data;
    struct Node *prev;
    struct Node *next;
};

typedef struct Node Node;

Node *front = NULL;
Node *rear = NULL;

void insertFront(int value)
{
    Node *newNode;

    newNode = (Node*)malloc(sizeof(Node));

    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = front;

    if(front == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        front->prev = newNode;
        front = newNode;
    }

    printf("%d inserted at front.\n", value);
}

void insertRear(int value)
{
    Node *newNode;

    newNode = (Node*)malloc(sizeof(Node));

    newNode->data = value;
    newNode->next = NULL;
    newNode->prev = rear;

    if(rear == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
```

```
        rear = newNode;
    }

    printf("%d inserted at rear.\n", value);
}

void deleteFront()
{
    Node *temp;

    if(front == NULL)
    {
        printf("Deque Underflow\n");
        return;
    }

    temp = front;

    printf("Deleted = %d\n", front->data);

    front = front->next;

    if(front == NULL)
    {
        rear = NULL;
    }
    else
    {
        front->prev = NULL;
    }

    free(temp);
}

void deleteRear()
{
    Node *temp;

    if(rear == NULL)
    {
        printf("Deque Underflow\n");
        return;
    }

    temp = rear;

    printf("Deleted = %d\n", rear->data);

    rear = rear->prev;

    if(rear == NULL)
    {
        front = NULL;
    }
    else
    {
        rear->next = NULL;
    }

    free(temp);
}

void display()
```

```
{
    Node *temp;

    if(front == NULL)
    {
        printf("Deque Empty\n");
        return;
    }

    temp = front;

    printf("Deque : ");

    while(temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    printf("\n");
}

int main()
{
    int choice;
    int value;

    while(1)
    {
        printf("\n===== DEQUE MENU =====\n");
        printf("1. Insert Front\n");
        printf("2. Insert Rear\n");
        printf("3. Delete Front\n");
        printf("4. Delete Rear\n");
        printf("5. Display\n");
        printf("6. Exit\n");

        printf("Enter Choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter Value: ");
                scanf("%d", &value);

                insertFront(value);
                break;

            case 2:
                printf("Enter Value: ");
                scanf("%d", &value);

                insertRear(value);
                break;

            case 3:
                deleteFront();
                break;

            case 4:
                deleteRear();
                break;
```

```
            case 5:
                display();
                break;

            case 6:
                exit(0);

            default:
                printf("Invalid Choice\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-17.c
$ ./a.out

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit
Enter Choice: 1
Enter Value: 10
10 inserted at front.

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit
Enter Choice: 2
Enter Value: 20
20 inserted at rear.

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit
Enter Choice: 1
Enter Value: 5
5 inserted at front.

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit
Enter Choice: 5
Deque : 5 10 20

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
```

```
5. Display
6. Exit
Enter Choice: 3
Deleted = 5

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit
Enter Choice: 5
Deque : 10 20

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit
Enter Choice: 4
Deleted = 20

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit
Enter Choice: 5
Deque : 10

===== DEQUE MENU =====
1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit
Enter Choice: 6
```

—

Assignment 3 : Problem 18

Problem Statement:
Implement a priority queue using

a. A single array
b. A single linked list
c. A 2D array
d. Multiple single linked list

ass-3-18a.c

```c
#include <stdio.h>
#include <stdlib.h>

#define MAX 100

typedef struct {
    int data;
    int priority;
} Node;

Node pq[MAX];
int size = 0;

void enqueue(int data, int priority) {
    if (size == MAX) {
        printf("Queue Overflow\n");
        return;
    }

    int i = size - 1;

    while (i >= 0 && pq[i].priority > priority) {
        pq[i + 1] = pq[i];
        i--;
    }

    pq[i + 1].data = data;
    pq[i + 1].priority = priority;
    size++;
    printf("Successfully inserted %d with priority %d\n", data, priority);
}

void dequeue() {
    if (size == 0) {
        printf("Queue Underflow\n");
        return;
    }

    printf("Deleted: %d (Priority: %d)\n", pq[0].data, pq[0].priority);

    for (int i = 1; i < size; i++)
        pq[i - 1] = pq[i];

    size--;
}

void display() {
    if (size == 0) {
        printf("Queue is empty.\n");
        return;
```

```
    }

    printf("Priority Queue: ");
    for (int i = 0; i < size; i++)
        printf("%d(P%d) ", pq[i].data, pq[i].priority);
    printf("\n");
}

int main() {
    int choice, data, priority;

    while (1) {
        printf("\n--- Priority Queue Menu ---\n");
        printf("1. Insert (Enqueue)\n");
        printf("2. Delete (Dequeue)\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                printf("Enter priority (lower number = higher priority): ");
                scanf("%d", &priority);
                enqueue(data, priority);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("Exiting program. Goodbye!\n");
                exit(0);

            default:
                printf("Invalid choice! Please enter a number between 1 and 4.\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-18a.c
$ ./a.out

--- Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 10
Enter priority (lower number = higher priority): 2
Successfully inserted 10 with priority 2
```

```
--- Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 5
Enter priority (lower number = higher priority): 0
Successfully inserted 5 with priority 0

--- Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 20
Enter priority (lower number = higher priority): 3
Successfully inserted 20 with priority 3

--- Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 3
Priority Queue: 5(P0) 10(P2) 20(P3)

--- Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 2
Deleted: 5 (Priority: 0)

--- Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 4
Exiting program. Goodbye!
```

ass-3-18b.c

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    int priority;
    struct Node *next;
};

struct Node *front = NULL;

void enqueue(int data, int priority) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory Allocation Failed! Queue Overflow\n");
        return;
    }
    newNode->data = data;
    newNode->priority = priority;
    newNode->next = NULL;

    if (front == NULL || priority < front->priority) {
```

```
        newNode->next = front;
        front = newNode;
        printf("Successfully inserted %d with priority %d\n", data, priority);
        return;
    }

    struct Node *temp = front;

    while (temp->next != NULL && temp->next->priority <= priority)
        temp = temp->next;

    newNode->next = temp->next;
    temp->next = newNode;
    printf("Successfully inserted %d with priority %d\n", data, priority);
}

void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow! Queue Empty\n");
        return;
    }

    struct Node *temp = front;

    printf("Deleted: %d (Priority: %d)\n", temp->data, temp->priority);

    front = front->next;

    free(temp);
}

void display() {
    if (front == NULL) {
        printf("Queue is empty.\n");
        return;
    }

    struct Node *temp = front;

    printf("Priority Queue: ");
    while (temp != NULL) {
        printf("%d(P%d) -> ", temp->data, temp->priority);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, data, priority;

    while (1) {
        printf("\n--- Priority Queue Menu (Linked List) ---\n");
        printf("1. Insert (Enqueue)\n");
        printf("2. Delete (Dequeue)\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
```

```c
                printf("Enter priority (lower number = higher priority): ");
                scanf("%d", &priority);
                enqueue(data, priority);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("Exiting program. Goodbye!\n");

                while (front != NULL) {
                    dequeue();
                }
                exit(0);

            default:
                printf("Invalid choice! Please enter a number between 1 and 4.\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-18b.c
$ ./a.out

--- Priority Queue Menu (Linked List) ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 10
Enter priority (lower number = higher priority): 2
Successfully inserted 10 with priority 2

--- Priority Queue Menu (Linked List) ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 5
Enter priority (lower number = higher priority): 0
Successfully inserted 5 with priority 0

--- Priority Queue Menu (Linked List) ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 20
Enter priority (lower number = higher priority): 3
Successfully inserted 20 with priority 3

--- Priority Queue Menu (Linked List) ---
1. Insert (Enqueue)
2. Delete (Dequeue)
```

```
3. Display
4. Exit
Enter your choice: 3
Priority Queue: 5(P0) -> 10(P2) -> 20(P3) -> NULL

--- Priority Queue Menu (Linked List) ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 2
Deleted: 5 (Priority: 0)

--- Priority Queue Menu (Linked List) ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 4
Exiting program. Goodbye!
Deleted: 10 (Priority: 2)
Deleted: 20 (Priority: 3)
```

ass-3-18c.c

```c
#include <stdio.h>
#include <stdlib.h>

#define PRIORITY 5
#define MAX 10

int pq[PRIORITY][MAX];
int rear[PRIORITY] = {0};

void enqueue(int data, int priority) {
    if (priority < 0 || priority >= PRIORITY) {
        printf("Invalid Priority! Please enter a priority between 0 and %d.\n", PRIORITY - 1);
        return;
    }

    if (rear[priority] == MAX) {
        printf("Queue Overflow for Priority %d\n", priority);
        return;
    }

    pq[priority][rear[priority]++] = data;
    printf("Successfully inserted %d at Priority %d\n", data, priority);
}

void dequeue() {
    for (int i = 0; i < PRIORITY; i++) {
        if (rear[i] > 0) {
            printf("Deleted: %d from Priority %d\n", pq[i][0], i);

            for (int j = 1; j < rear[i]; j++)
                pq[i][j - 1] = pq[i][j];

            rear[i]--;
            return;
        }
    }

    printf("Queue Underflow! All priority queues are empty.\n");
}
```

```c
void display() {
    int isEmpty = 1;
    printf("\n--- Current Priority Queue Status ---\n");
    for (int i = 0; i < PRIORITY; i++) {
        if (rear[i] > 0) {
            isEmpty = 0;
            printf("Priority %d: ", i);
            for (int j = 0; j < rear[i]; j++)
                printf("%d ", pq[i][j]);
            printf("\n");
        } else {
            printf("Priority %d: [Empty]\n", i);
        }
    }
    if (isEmpty) {
        printf("Entire Queue is completely empty.\n");
    }
}

int main() {
    int choice, data, priority;

    while (1) {
        printf("\n--- Multi-Queue Priority Queue Menu ---\n");
        printf("1. Insert (Enqueue)\n");
        printf("2. Delete (Dequeue)\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                printf("Enter priority (0 to %d, where 0 is highest): ", PRIORITY - 1);
                scanf("%d", &priority);
                enqueue(data, priority);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("Exiting program. Goodbye!\n");
                exit(0);

            default:
                printf("Invalid choice! Please enter a number between 1 and 4.\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-18c.c
$ ./a.out

--- Multi-Queue Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 10
Enter priority (0 to 4, where 0 is highest): 2
Successfully inserted 10 at Priority 2

--- Multi-Queue Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 5
Enter priority (0 to 4, where 0 is highest): 0
Successfully inserted 5 at Priority 0

--- Multi-Queue Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 20
Enter priority (0 to 4, where 0 is highest): 3
Successfully inserted 20 at Priority 3

--- Multi-Queue Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 3

--- Current Priority Queue Status ---
Priority 0: 5
Priority 1: [Empty]
Priority 2: 10
Priority 3: 20
Priority 4: [Empty]

--- Multi-Queue Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 2
Deleted: 5 from Priority 0

--- Multi-Queue Priority Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 4
Exiting program. Goodbye!
```

ass-3-18d.c

```c
#include <stdio.h>
#include <stdlib.h>

#define PRIORITY 5
```

```c
struct Node {
    int data;
    struct Node *next;
};

struct Node *front[PRIORITY] = {NULL};
struct Node *rear[PRIORITY] = {NULL};

void enqueue(int data, int priority) {
    if (priority < 0 || priority >= PRIORITY) {
        printf("Invalid Priority! Please enter a value between 0 and %d.\n", PRIORITY - 1);
        return;
    }

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory Allocation Failed! Queue Overflow\n");
        return;
    }
    newNode->data = data;
    newNode->next = NULL;

    if (front[priority] == NULL) {
        front[priority] = rear[priority] = newNode;
    } else {
        rear[priority]->next = newNode;
        rear[priority] = newNode;
    }
    printf("Successfully inserted %d at Priority %d\n", data, priority);
}

void dequeue() {
    for (int i = 0; i < PRIORITY; i++) {
        if (front[i] != NULL) {
            struct Node *temp = front[i];

            printf("Deleted: %d from Priority %d\n", temp->data, i);

            front[i] = front[i]->next;

            if (front[i] == NULL)
                rear[i] = NULL;

            free(temp);
            return;
        }
    }

    printf("Queue Underflow! All priority queues are empty.\n");
}

void display() {
    int isEmpty = 1;
    printf("\n--- Current Priority Queue Status ---\n");
    for (int i = 0; i < PRIORITY; i++) {
        struct Node *temp = front[i];

        if (temp != NULL) {
            isEmpty = 0;
            printf("Priority %d: ", i);
            while (temp != NULL) {
                printf("%d -> ", temp->data);
                temp = temp->next;
```

```
            }
            printf("NULL\n");
        } else {
            printf("Priority %d: [Empty]\n", i);
        }
    }
    if (isEmpty) {
        printf("Entire Queue is completely empty.\n");
    }
}

int main() {
    int choice, data, priority;

    while (1) {
        printf("\n--- Linked List Multi-Queue Menu ---\n");
        printf("1. Insert (Enqueue)\n");
        printf("2. Delete (Dequeue)\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                printf("Enter priority (0 to %d, where 0 is highest): ", PRIORITY - 1);
                scanf("%d", &priority);
                enqueue(data, priority);
                break;

            case 2:
                dequeue();
                break;

            case 3:
                display();
                break;

            case 4:
                printf("Exiting program. Cleaning up memory...\n");
                for (int i = 0; i < PRIORITY; i++) {
                    while (front[i] != NULL) {
                        struct Node *temp = front[i];
                        front[i] = front[i]->next;
                        free(temp);
                    }
                }
                printf("Goodbye!\n");
                exit(0);

            default:
                printf("Invalid choice! Please enter a number between 1 and 4.\n");
        }
    }

    return 0;
}
```

Output

```
$ gcc ass-3-18d.c
$ ./a.out

--- Linked List Multi-Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 10
Enter priority (0 to 4, where 0 is highest): 2
Successfully inserted 10 at Priority 2

--- Linked List Multi-Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 5
Enter priority (0 to 4, where 0 is highest): 0
Successfully inserted 5 at Priority 0

--- Linked List Multi-Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 1
Enter data: 20
Enter priority (0 to 4, where 0 is highest): 3
Successfully inserted 20 at Priority 3

--- Linked List Multi-Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 3

--- Current Priority Queue Status ---
Priority 0: 5 -> NULL
Priority 1: [Empty]
Priority 2: 10 -> NULL
Priority 3: 20 -> NULL
Priority 4: [Empty]

--- Linked List Multi-Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 2
Deleted: 5 from Priority 0

--- Linked List Multi-Queue Menu ---
1. Insert (Enqueue)
2. Delete (Dequeue)
3. Display
4. Exit
Enter your choice: 4
Exiting program. Cleaning up memory...
Goodbye!
```

—